Why a browser needs an agent

Browser Use: How LLM Agents Drive the Web

Week 1, Lecture 1 · Summer 2026

What we'll cover

- Why the browser is the universal interface
- Scripted automation and where it breaks
- What browser-use is: task, LLM, browser
- The project: founders, origin, scale
- Where today fits in the ten-week arc

The browser as the universal interface

The web is where things happen

- Banking, filing taxes, booking travel, ordering supplies: all in a browser
- APIs exist for some tasks; for most, the browser is the only interface
- "Screen scraping" has existed for decades, but it is brittle
- The agent approach: describe the task in words, let the system figure out the clicks

Scripted automation breaks on change

- A Playwright or Selenium script names a CSS selector or XPath
- The site updates its layout: the script breaks
- Maintaining selectors is continuous work; the site's authors do not know you exist
- The cost compounds: 10 scripts across 10 sites means 10 maintenance burdens

The agent alternative

- Give the agent a task in plain English: *"Find the price of item X on this website"*
- The agent reads the page, decides what to click, clicks it, and reads again
- If the layout changes, the agent adapts; there are no hardcoded selectors
- The trade-off: slower, costs LLM tokens, less predictable than a tight script

What browser-use is

browser-use in one paragraph

- Open-source Python framework: github.com/browser-use/browser-use
- You provide a **task** (plain English) and an **LLM** (OpenAI, Anthropic, Google, or others)
- browser-use opens a real browser, reads the page, asks the LLM what to do, does it
- The loop runs until the task is done or a step limit is hit
- A hosted cloud version exists alongside the open-source library

The minimal API

```python
import asyncio
from browser_use import Agent, ChatOpenAI

async def main():
    agent = Agent(
        task="Find the number 1 post on Show HN",
        llm=ChatOpenAI(model="gpt-4.1-mini"),
    )
    history = await agent.run()
    print(history.final_result())

asyncio.run(main())
```

(browser-use team, 2026)

The people and the project

Origin story

- Magnus Müller and Gregor Žunič met at ETH Zurich
- Launched on Hacker News in November 2024: *"Open-source browser alternative for Computer Use for any LLM"* (Müller, 2024)
- Within months: tens of thousands of GitHub stars; one of the fastest-growing open-source AI agent projects
- Went through Y Combinator W25; seed round led by Felicis Ventures (Felicis, 2025)

Two products, one library

	What it is
Open-source library	`pip install browser-use` (the Python agent you control)
Hosted cloud	Runs agents in the cloud; no local browser required
CLI	Run tasks from the command line

Both the library and the cloud share the same core architecture.

> "Browser Use enables AI agents to interact with the web the way humans do: visually, contextually, and reliably."

Astasia Myers, Felicis Ventures. "Investing in Browser Use" (2025)

What's next

- **Lecture 2 (this week):** The perceive-decide-act loop step by step
- **Reading (this week):** Full chapter on what browser-use is, the loop, and how to read a run's history
- **Section (this week):** Install browser-use, run three tasks, read each history
- **Week 2:** How the agent actually sees a web page (DOM, accessibility tree, numbered elements)

Take-aways

- The browser is the universal interface for tasks that have no API
- Scripted automation is brittle; agent-driven automation trades predictability for adaptability
- browser-use: a task, an LLM, and a browser: three ingredients, one loop
- The project launched in late 2024 and grew rapidly; ETH Zurich founders, YC W25